



Student Performance AI Agent

Technical Functional Requirement Specification (TFRS)

Primary Interfaces: Web Dashboard + Integrated AI Chatbot

Technology Stack: Gemini/Grok API, LangChain, LangGraph, RAG, FastAPI, PostgreSQL/SQLite, ChromaDB, Streamlit/Web UI

Project Duration: Feb 16, 2026 – Feb 28, 2026

1. Project Overview

Purpose

The Student KPI Tracking & Performance Intelligence AI Agent is an intelligent web-based system designed to track, analyze, and improve student performance using Key Performance Indicators (KPIs) beyond academic CGPA.

The system enables institutions to measure career readiness and performance based on multiple dimensions such as internships, certifications, hackathons, publications, workshops, and technical activities.

The system includes:

- Web Dashboard for visualization and management
- Integrated AI Chatbot for intelligent interaction
- KPI Intelligence Engine for score calculation
- Agentic AI workflow using LangGraph
- RAG-based recommendation system

This system provides real-time academic intelligence and performance analytics.

2. System Objectives

The system must:

- Track student KPI data across multiple categories
- Calculate overall KPI score and career readiness score
- Provide performance dashboards for students, faculty, and HOD



- Provide AI-driven personalized recommendations
 - Enable intelligent chatbot interaction
 - Generate class-level and department-level analytics
 - Enable institutional decision-making based on KPI insights
-

3. System Architecture

High-Level Architecture

```
Web Dashboard (Streamlit / Web UI)
    ↓
Integrated Web Chatbot Interface
    ↓
FastAPI Backend (Application Layer)
    ↓
LangGraph Agent Workflow Engine
    ↓
LLM Integration Layer (Gemini / Grok via LangChain)
    ↓
KPI Intelligence Engine
    ↓
Relational Database (PostgreSQL / SQLite)
    ↓
Vector Database (ChromaDB for RAG)
```

4. User Roles and Capabilities

Student Role

Students can:

- View KPI score
- View KPI breakdown by category
- View career readiness score
- View rank within class
- Receive AI recommendations
- Ask questions via chatbot

Example chatbot queries:

```
Show my KPI score
What should I improve?
Am I ready for placement?
```



Faculty Role

Faculty can:

- View KPI data for students in their class
- Identify weak students
- Identify top performers
- View class KPI analytics
- Ask chatbot for performance insights

Example:

Show students with low KPI score

HOD Role

HOD can:

- View department KPI analytics
 - Compare sections
 - View placement readiness trends
 - Generate reports
-

Admin Role

Admin can:

- Add student KPI data
 - Update KPI records
 - Upload KPI data via CSV
 - Manage system data
-

5. KPI Categories to be Tracked

The system must track the following KPI parameters:

- Project funding participation



- Association activities
- Alumni lecture participation
- Value-added courses
- Technical presentations and contests
- Hackathon participation
- Workshop participation
- Awards and prizes
- Mini project participation
- Club activities
- Industrial visits
- Internship participation
- Certification courses (IIT Spoken Tutorials, NPTEL)
- Journal publications
- Conference publications

6. Functional Modules

Module 1: KPI Data Management Module

Function

Store and manage KPI data.

Input Methods

- Manual entry via dashboard
- CSV upload
- Admin update

Data Stored

- Student profile
- KPI category data
- Activity counts

Module 2: KPI Score Calculation Engine



Function

Calculate overall KPI score.

Example Calculation Formula

```
KPI Score =  
Internship × 25 +  
Certification × 15 +  
Hackathon × 20 +  
Publication × 25 +  
Workshop × 5 +  
Projects × 10
```

Weights configurable.

Output

KPI Score: 72/100

Module 3: Career Readiness Prediction Module

Function

Predict student placement readiness based on KPI score.

Example Logic

```
Score > 80 → High readiness  
Score 50-80 → Medium readiness  
Score < 50 → Low readiness
```

Module 4: Web Dashboard Module

The dashboard must include:

Student Dashboard

Displays:



- KPI score
 - KPI category breakdown
 - Career readiness score
 - Performance trends
 - AI recommendations
-

Faculty Dashboard

Displays:

- Student KPI list
 - Weak students
 - Strong students
 - Class performance analytics
-

HOD Dashboard

Displays:

- Department KPI analytics
 - Section comparison
 - KPI trends
-

Admin Dashboard

Displays:

- KPI data management interface
 - Upload and edit functionality
-

Module 5: Integrated AI Chatbot Module

The chatbot must be integrated into the web dashboard.

Chatbot Capabilities



Students can ask:

```
Show my KPI score
What should I improve?
What internships should I focus on?
```

Faculty can ask:

```
Show weak students
```

HOD can ask:

```
Show department performance summary
```

Module 6: AI Recommendation Engine (LangChain + RAG)

The system must generate recommendations based on KPI data.

Example output:

```
Recommendation:
You should participate in internships and hackathons to improve career
readiness.
```

Uses:

- LLM (Gemini or Grok)
 - Vector DB knowledge retrieval
-

Module 7: LangGraph Agent Workflow Module

LangGraph manages system workflow.

Workflow States

```
Data ingestion
KPI calculation
Career readiness prediction
Recommendation generation
```



Dashboard update
Chatbot response generation

LangGraph orchestrates processing.

Module 8: KPI Knowledge Base Module (Vector Database)

Vector DB stores:

- KPI insights
- Recommendations
- Career advice knowledge

Supports RAG chatbot responses.

7. Backend API Requirements (FastAPI)

Required endpoints:

```
GET /student/{id}
GET /student/{id}/kpi_score
GET /student/{id}/recommendation
GET /class/{section}/summary
POST /kpi/add
POST /kpi/update
POST /chatbot/query
```

8. Database Schema Requirements

Students Table

```
student_id
name
department
section
year
```



KPI Table

```
student_id  
internships  
certifications  
hackathons  
publications  
workshops  
projects  
club_activities  
industrial_visits
```

Score Table

```
student_id  
kpi_score  
career_readiness_score  
last_updated
```

9. Vector Database Requirements (ChromaDB)

Store:

- KPI performance insights
- Recommendations
- Career readiness analysis

Used for chatbot RAG retrieval.

10. Technical Implementation Approach

Students must implement in this order:

- Step 1: Setup database and backend
- Step 2: Implement KPI data ingestion
- Step 3: Implement KPI calculation engine
- Step 4: Implement career readiness prediction
- Step 5: Build web dashboard
- Step 6: Integrate LLM using LangChain



Step 7: Implement vector DB using ChromaDB

Step 8: Implement LangGraph workflow

Step 9: Integrate chatbot into dashboard

Step 10: Final testing and optimization

11. Recommended Project Folder Structure

```
student-kpi-agent/  
├── backend/  
│   ├── main.py  
│   └── api/  
├── agent/  
│   ├── langgraph_workflow.py  
│   └── recommendation_engine.py  
├── database/  
│   └── models.py  
├── dashboard/  
│   └── streamlit_app.py  
├── chatbot/  
│   └── web_chatbot.py  
├── vector_db/  
├── requirements.txt  
└── README.md
```

12. Stretch Goals (Advanced Features)

Students can implement:

- Student ranking leaderboard
 - Placement readiness prediction using ML
 - Email alerts for weak students
 - Automated mentoring recommendations
 - KPI trend prediction
-

13. Deliverables



Teams must submit:

- Working web dashboard
- Integrated chatbot
- KPI scoring engine
- Backend API
- Vector DB integration
- GitHub repository
- Demo video
- Documentation

14. Final Expected Outcome

The system must provide:

- Full KPI tracking system
- AI-powered recommendations
- Career readiness prediction
- Interactive web dashboard
- Integrated AI chatbot
- Faculty and HOD analytics

Day-Wise Execution Plan (Web Dashboard + Chatbot)

Duration: Feb 16, 2026 – Feb 28, 2026

Final Deliverable: Fully working KPI Dashboard + AI Chatbot + Agent Workflow

Execution Plan Table

Date	Day	Objective	Tasks	Expected Deliverable
Feb 16	Mon	Project Setup & Architecture Initialization	Create GitHub repo, install Python, FastAPI, Streamlit, LangChain, LangGraph, ChromaDB. Create project	Project initialized, environment ready



Date	Day	Objective	Tasks	Expected Deliverable
			folder structure. Setup virtual environment.	
Feb 17	Tue	Database Design & Implementation	Create database schema: students table, KPI table, score table. Implement database connection module. Insert sample student records.	Database created and functional
Feb 18	Wed	KPI Data Ingestion Module	Build API to add KPI data. Create CSV upload functionality. Test storing KPI data.	KPI data successfully stored
Feb 19	Thu	KPI Calculation Engine	Implement KPI score calculation logic. Implement career readiness prediction logic. Store calculated score in database.	KPI score calculation working
Feb 20	Fri	Backend API Development	Implement FastAPI endpoints: student data retrieval, KPI score retrieval, recommendation endpoint. Test API using Postman.	Backend APIs functional
Feb 21	Sat	Web Dashboard – Student View	Build Streamlit dashboard. Display KPI score, KPI breakdown, career readiness score. Connect dashboard to backend API.	Student dashboard working
Feb 22	Sun	Faculty & HOD Dashboard	Build faculty dashboard view. Display student list, KPI score, weak students. Add filters by section/department.	Faculty/HOD dashboard working
Feb 23	Mon	Vector Database Integration (ChromaDB)	Setup ChromaDB. Store KPI insights and recommendations. Implement embedding generation using Gemini/Grok.	Vector DB integrated
Feb 24	Tue	Chatbot Backend Implementation	Build chatbot API endpoint /chatbot/query. Integrate LangChain with Gemini/Grok. Implement RAG retrieval using vector DB.	Chatbot backend working



Date	Day	Objective	Tasks	Expected Deliverable
Feb 25	Wed	Chatbot UI Integration (Web Dashboard)	Add chatbot panel to dashboard. Connect chatbot UI with backend API. Test chatbot queries and responses.	Chatbot integrated into dashboard
Feb 26	Thu	LangGraph Agent Workflow Integration	Implement LangGraph workflow: data ingestion → KPI calculation → recommendation generation → chatbot response.	Agent workflow functional
Feb 27	Fri	Recommendation Engine Enhancement & Testing	Improve recommendation logic. Test multiple student scenarios. Validate chatbot recommendations. Fix bugs.	Recommendation engine working
Feb 28	Sat	Final Integration, Testing & Submission	Perform end-to-end testing. Optimize dashboard and chatbot. Complete documentation. Record demo video. Submit GitHub repo.	Final working system ready

Mandatory Milestone Targets

By Feb 19

Teams must complete:

- Database setup
- KPI ingestion
- KPI calculation engine

By Feb 22

Teams must complete:

- Student dashboard
- Faculty dashboard



By Feb 25

Teams must complete:

- Vector DB integration
 - Chatbot backend
 - Chatbot UI integration
-

By Feb 28

Teams must complete:

- Full dashboard
 - AI chatbot
 - LangGraph workflow
 - Recommendation engine
 - Final system integration
-

Daily Progress Reporting Format (Post in Team Channel)

Teams must post update daily:

Date:

Tasks Completed:

GitHub Commit Link:

Tasks In Progress:

Blockers (if any):

Plan for Tomorrow:

Final Deliverables Checklist (Feb 28)

Each team must submit:

- Working Web Dashboard
- Integrated AI Chatbot
- KPI Score Calculation Engine
- Career Readiness Prediction



- LangGraph Agent Workflow
- Vector DB Integration
- GitHub Repository
- Demo Video (5 minutes)
- Documentation

Expected Final System Capabilities

By Feb 28, the system must:

- Track student KPI performance
- Calculate KPI score automatically
- Predict career readiness
- Provide dashboard analytics
- Provide AI chatbot interaction
- Generate recommendations

Recommended Daily Time Allocation

Activity	Recommended Time
Backend development	2 hours/day
Dashboard development	1.5 hours/day
Chatbot integration	1.5 hours/day
Testing & debugging	1 hour/day